


# Who Can Turn Off Your Ethereum App?

## A Full Stack Walk Away Test

**James Carnley**

Ethereum File System




Building Cypherpunk Apps

# Why make an app you can walk away from?

**What happens when an app disappears?**

Users lose their work and trust

Other developers can't rely on that functionality

Communities splinter

# Software worth building on

**For people**

Keep years of useful work

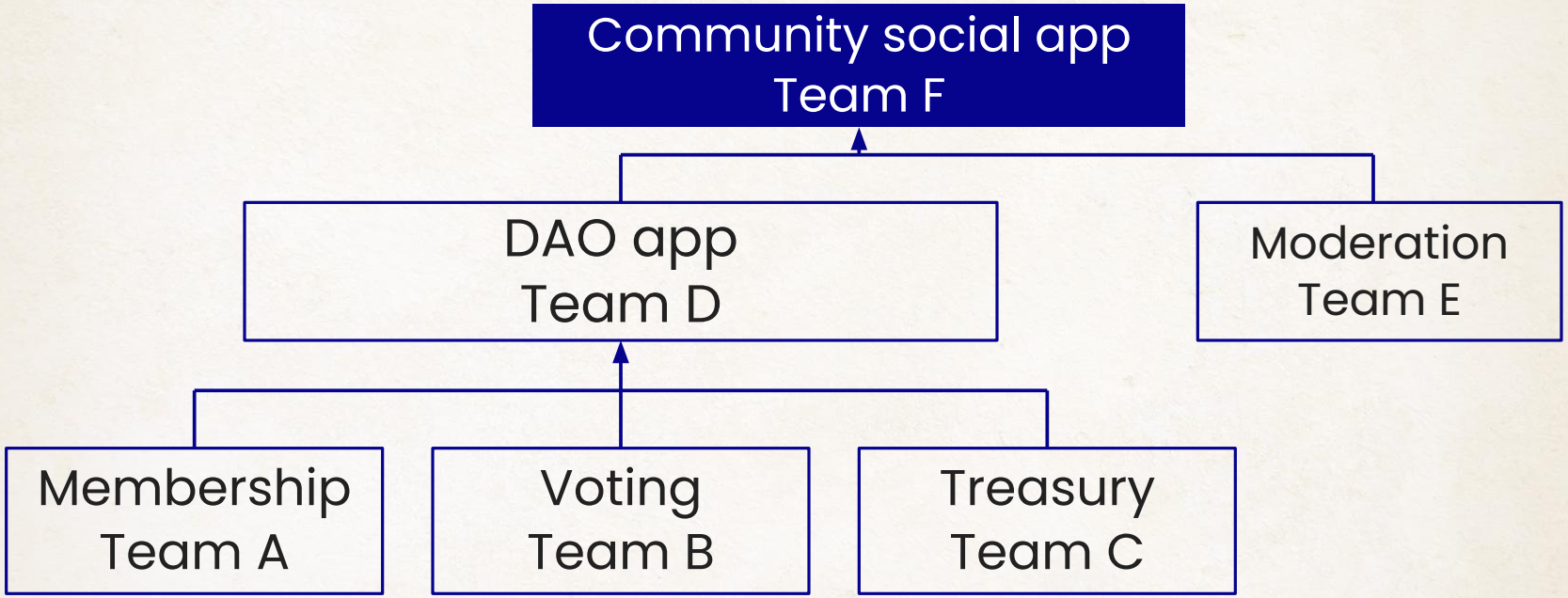
Choose better tools

**For builders**

Reuse dependable capabilities

Build bigger things together

# Hyperstructures: build on each other




Independent teams work together via stable interfaces

# A walk away test

Single points of failure and control

**The team disappears**

Can users keep working?

Can others run the services?

**Control becomes hostile**

Can users keep their work?

Can they keep the rules they chose?

**TEST**

Choose one important user task.
Try it without your team’s services or help.

# The application stack

## Interface

- Names and discovery
- Runnable software
- Accounts and recovery

## Data

- Storage and formats
- State and indexes
- Background work

## Rules

- Contracts and governance
- Network and settlement




Interface

# Finding and using the app

Find a saved release

Run it without your servers

Recover access without your help

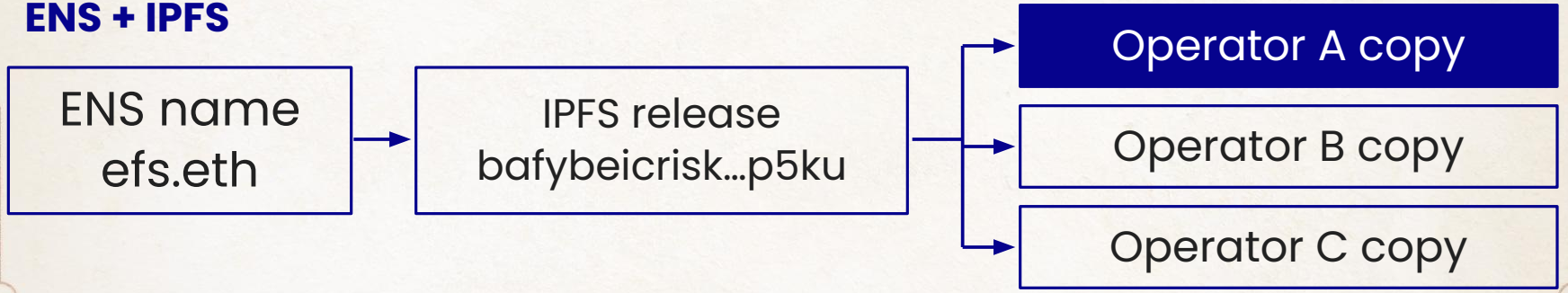
**Can people find and use your app?**

# More than one way into the app

## DNS + HTTP




## ENS + IPFS




IPFS gateways: efs.eth.limo (or .link) or your own IPFS node

**TEST** Block the usual website. Can you open a trusted release through another route?

# A complete runnable release

Keep every file the client needs to start

**Incomplete release**

index.html on IPFS

↓

Team server: app.js
Offline

Client cannot start

**Complete release**

IPFS release
index.html
app.js + styles + assets

↓

Client starts

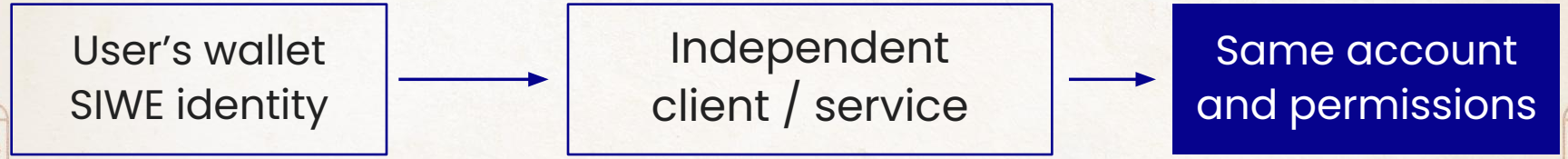
**TEST** Block your team's server, then reload the client.
Does it start?

# Access without the company

The client starts. Can the user actually get in?




**An account another service can recognize**




**TEST** Disable company login. Can the user complete the same action through another client?




Data

# Continuing the work

**Is the data portable?**

Keep the files and their meaning

Use them through another app

Let others run the supporting services

# Data another app can understand

## Example shared record

```
{
  id: note-17
  author: 0xDEADBEEF…
  text: Meeting at 10
  body: Review proposals.
  file: agenda.md
  fileRevision: 2
}
```

## Shared meaning

What each field means

Who can change it

How versions fit together

Who can read and decrypt it



Shared schemas with EAS. Documented exports from your database.

**TEST**

Hand over an export and its format.
Can another developer read it without asking you?

# Different tools. The same work.

## Notes

MY NOTES

Meeting

Ideas

Projects

Archive

Edit Format Attach

**Meeting at 10**

0xDEADBEEF…

Review proposals.

agenda.md revision 2

Saved as note-17

## Community

Following Community

**0xDEADBEEF…**

Shared note / note-17

**Meeting at 10**

Review proposals.

Open agenda.md (rev 2)

Reply Share Save

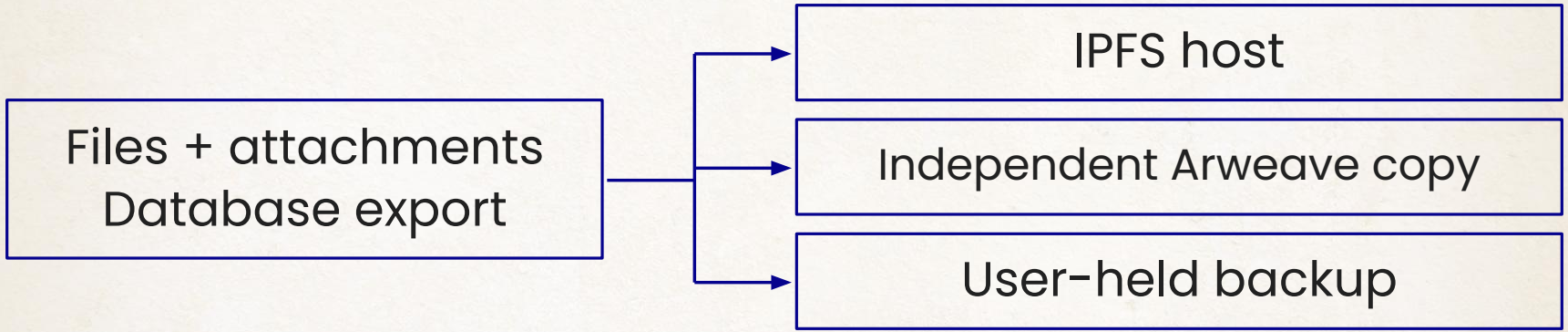
Same record: note-17 and agenda.md (revision 2)

**TEST**

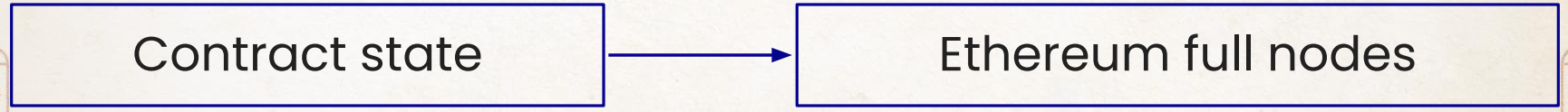
Open the same record in another app.
Can the user continue their work?

# Who keeps a usable copy?

## Files and databases




## Onchain records

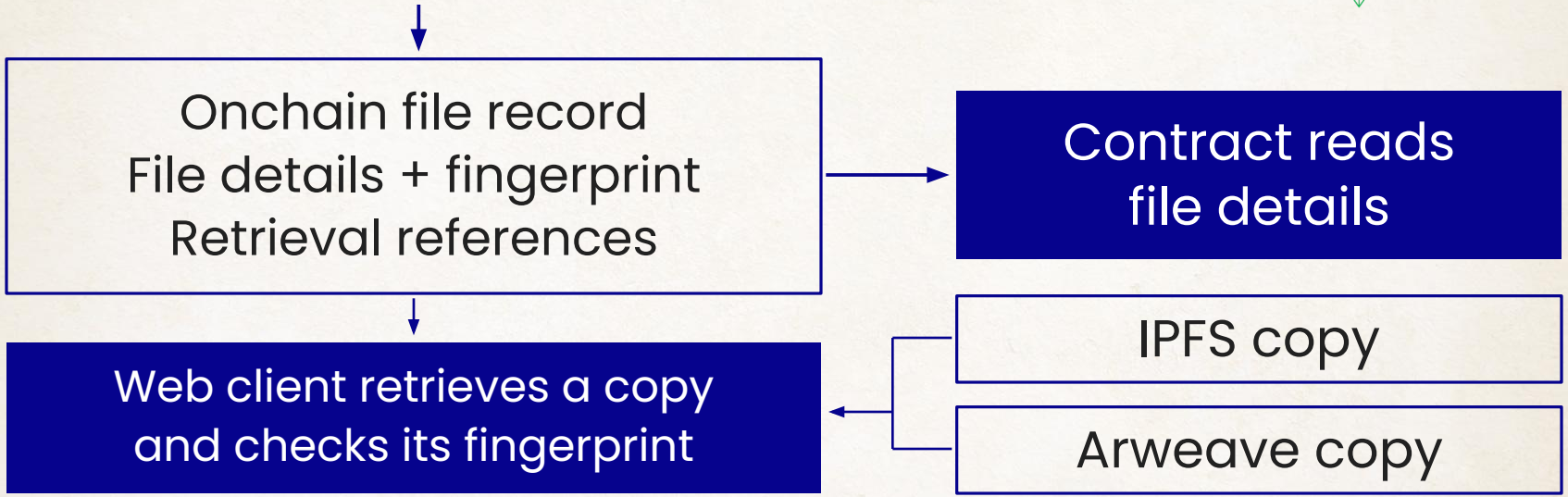



**TEST** Block the primary storage provider.
Can you recover the record and attachments elsewhere?

# EFS: find a file and check a copy

Example path: /notes/meeting.md

/EFS/




**TEST** Fetch from another source. Does its fingerprint match the accepted file record?

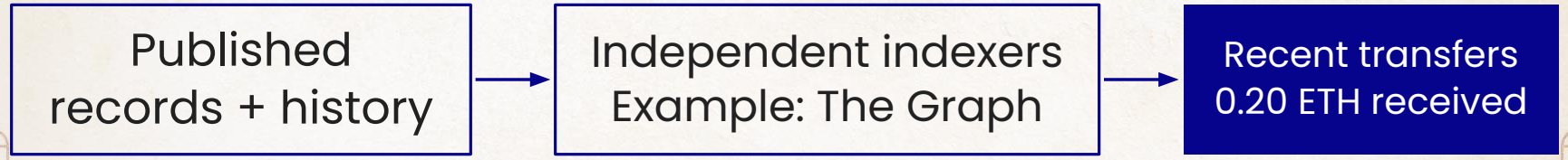
# The data survives. Can your app read it?

## Read a wallet balance




Light clients such as Helios can verify supported chain reads.

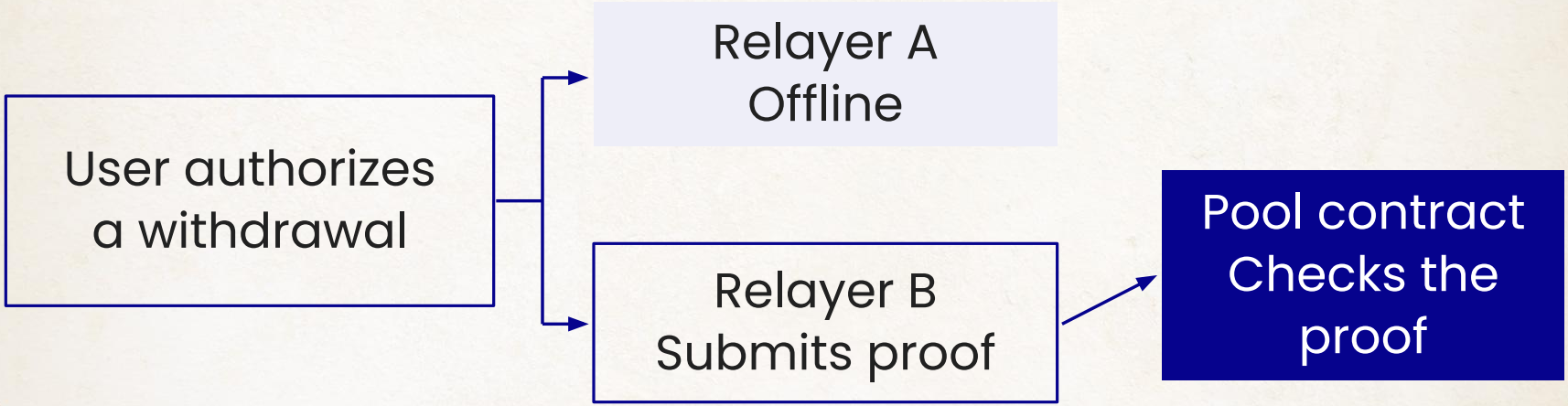
## Show a transaction history




**TEST** Switch data providers. Can users still find their records and latest changes?

# Who sends the next transaction?

Tornado Cash: replaceable relayers




**TEST** Stop your transaction service. Can another operator complete the user's request?




Rules

# Keeping dependable rules

Stable core contracts

User-controlled app upgrades

Clear rules for shared governance

**Who chooses the changes?**

**CHECK** Who can change the rules? What can users keep using if those powers are abused?

# Immutable contracts

UNIX mindset: Do one thing and do it well

## A stable core

Keep contract code fixed

Let other apps build on it

## Room to improve

Deploy a new version

Let users choose to migrate

Uniswap v2 core: fixed code, shared exchange logic

**TEST**

Keep the old app release and contract address.
Can users complete the task without upgrading?

# The network beneath the app

**Ethereum and L1**

Use another node

Keep required history

Check inclusion and finality

**L2 and settlement**

Who orders transactions?

Can users submit through L1?

How can users withdraw?

**TEST** Rehearse a documented network outage.
Can the user complete the recovery steps?

# Could someone run it without you?

No private help from the team

## Public handoff

Software + license
Data + instructions
Configuration + costs

→ **Independent operator**

Their own keys and hosting

→ **User's task completed**

A working service users can choose

**TEST** Give an outsider the docs. Can they run it and complete the task without contacting you?




Putting this into practice

# A walk away test for your app

# Your walk away checklist

## Interface

Open a saved release

Recover access and keys

## Data

Restore usable records

Switch clients and providers

Run essential background jobs

## Rules

Identify who can change rules

Keep using accepted rules

Try the chain’s outage or exit path

**TEST**

Remove one team dependency. Repeat the user’s task. Improve what stops it, then try again.

# A walk away test in practice

Test a saved export with the original app blocked.

## 1. Import saved data

**Meeting at 10**

Review proposals.

agenda.md
Attachment missing

**Task blocked**

→

## 2. Fix the export

**Complete export**

notes.json
agenda.md

Reconnect. Save all files.
Block the app again.

→

## 3. Import again

**Meeting at 10**

Review proposals.

agenda.md opens
Review proposals

**Task completed**

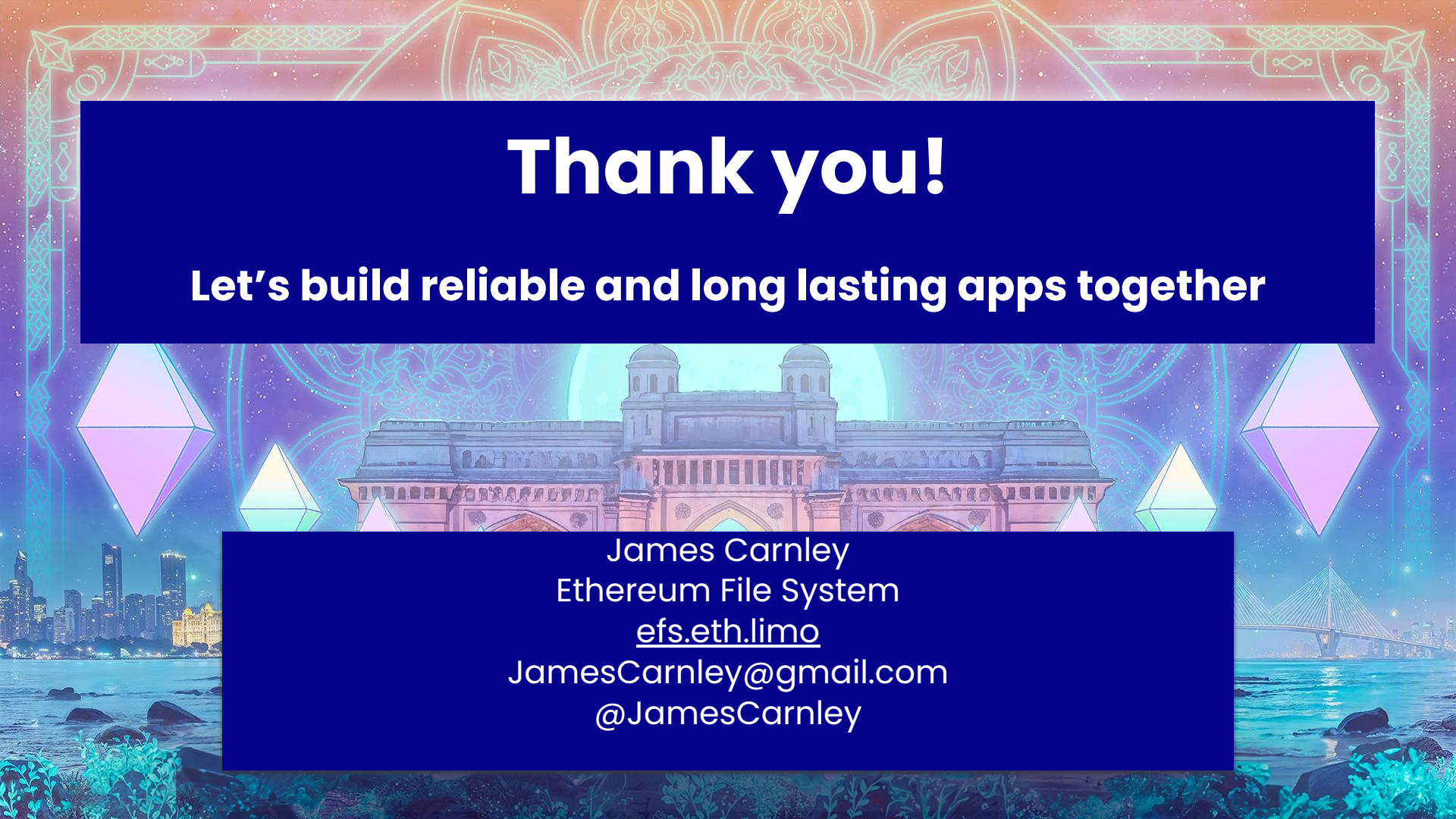
**TEST**

Import a saved export into another app.
Can you open the note and its attachment?

# Our work can outlive us

# Thank you!

**Let’s build reliable and long lasting apps together**



James Carnley
Ethereum File System
efs.eth.limo
JamesCarnley@gmail.com
@JamesCarnley